

Seedcraft Sequences

by hand

Draft v0 · test seeds only · github.com/ndssigner/seedcraft

A **Seedcraft sequence** stores a BIP-39 seed as a row of objects — beads, charms, bricks... — in a fixed order. Behind it there is a **CompactSeedQR**, the QR code of the seed that SeedSigner and NDS-Signer scan. This booklet lets you go from the sequence to that QR code, and from the QR code to a sequence, **with a pencil and nothing else**: no computer, no arithmetic. Then scan the QR code you drew with an air-gapped signer.

Before you start. Whoever has the sequence, or the QR code you draw, has the seed. Work alone, and burn or shred the worksheets afterwards. The format is a **draft**: practise with test seeds, not with real funds.

What is in the sequence

From the start (the clasp or a knot, or one end of an open carrier):

1. **The dictionary**: N objects, all different. The first one means 0, the second 1, and so on up to N – 1. N is 2, 4, 8, 16 or 32.
2. **The symbols**: one object per symbol. Each one stands for k bits (0s and 1s):

N (kinds of objects)	2	4	8	16	32
k (bits per object)	1	2	3	4	5
symbols, 12 words	213	107	71	54	43
symbols, 24 words	364	182	122	91	73

The bits, in order, are the QR code's **data cells** read row by row, left to right, like writing: 208 cells for 12 words (21×21 squares) or 359 for 24 words (25×25). Then come 5 header bits: 00 (the format version) and 3 bits with the QR code's **mask** number. The bits left over at the end are zeros and mean nothing.

The templates at the end of this booklet have every fixed part of the QR code already drawn — the three big squares, the dotted lines, and the 15 **format cells** that depend on the mask — so there is one template per size and mask. You only fill in the white squares: **1 = dark, 0 = leave it white**.

A. From a sequence to the QR code

- 1 **Find the start.** At one end, the first objects are all different from each other: that is the dictionary. Count them: that is N. If both ends look like that, try one; if the QR code does not scan, try from the other end.
- 2 **Copy the dictionary** onto the worksheet: draw or name each object next to its number (0, 1, 2...).
- 3 **Write the numbers.** For each object after the dictionary, write its number in the next box of the worksheet. Count the boxes: with the table above, that tells 12 words (21×21) or 24 words (25×25).
- 4 **Turn each number into bits** with the conversion table (column k), and write them under it.
- 5 **Read the header.** Count 208 bits (12 words) or 359 bits (24 words) from the start; the next five should be 00 followed by the mask's three bits. Take the template for that size and mask.
- 6 **Fill the white squares** row by row, left to right: 1 = colour it dark (a black felt-tip works best), 0 = leave it. The number at the right of each row says how many white squares it has: tick each row off as you finish it.

- 7 **Scan it** with SeedSigner or NDS-Signer (“Scan a SeedQR”). One misread object is repaired by the QR code itself.

Instead of drawing, you can lay the QR code out on a fusible-bead pegboard (usually 29×29 pegs) or a Lego baseplate, with dark and light pieces.

If the sequence was made with a PIN, the QR code you draw is the **scrambled** one: a signer reads it as a different, empty wallet. Undoing the PIN needs a computation that cannot be done by hand: read the sequence with the Seedcraft web tool, offline, and type the PIN there.

B. From a SeedQR to a sequence

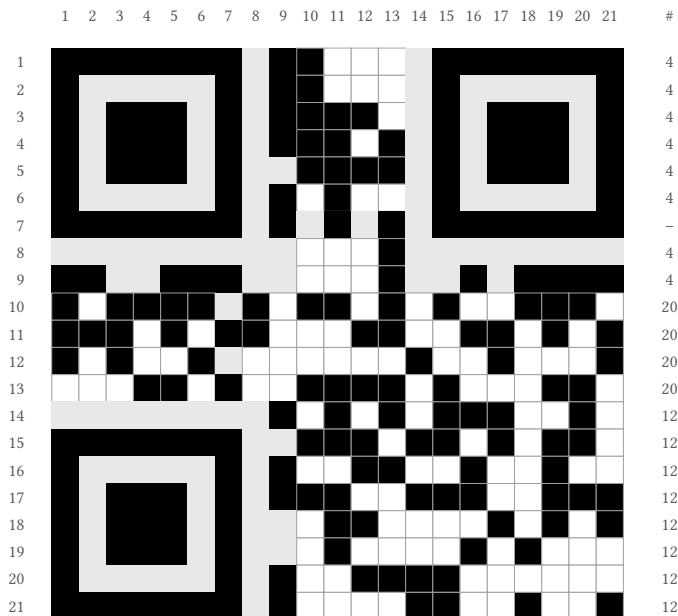
- 1 **Show the CompactSeedQR** on your signer: in SeedSigner, Backup Seed $\rightarrow$ Export as SeedQR $\rightarrow$ Compact (it shows it zone by zone so you can copy it); NDS-Signer has a SeedQR map. It must be the **Compact** kind: 21×21 for 12 words, 25×25 for 24.
- 2 **Find its mask.** Compare row 9 and column 9 next to its top-left big square with the drawings of the mask table: one matches. Take the template for that size and mask: its grey and black squares match the code too.
- 3 **Read the data cells** — the squares that are white on the template — row by row, left to right: dark = 1, light = 0. Write the bits in groups of k in the worksheet boxes.
- 4 **Add the header:** 00 and the mask’s three bits. Complete the last group with zeros.
- 5 **Turn each group into its number** with the conversion table, and string the objects: first the dictionary (the objects for 0, 1, ... $N - 1$), then one object per number.

A PIN cannot be added by hand. If your signer chose a different mask from the one Seedcraft’s software would pick, your sequence is still valid: readers try every mask.

Worked example

The NDS-Signer **test seed** (public, never use it for real funds), 12 words, with N = 16 kinds of objects (k = 4), without a PIN:

height demise useless trap grow lion found off key clown transfer enroll



Its CompactSeedQR uses **mask 4** (header 00100). Squares with a thin border are data cells; the rest is the template.

The first rows with data cells are rows 1–3, with 4 cells each:

row	bits	number	object
1	1000	8	8
2	1000	8	8
3	1110	14	14

Bits 209–213 are the header: 00100. The last 3 bits are padding zeros.

The whole sequence: the dictionary (the 16 objects, here numbered 0–15), then the 54 symbols — 70 objects. Check your own reading against it, five at a time:

Dictionary: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Tables

Number → bits

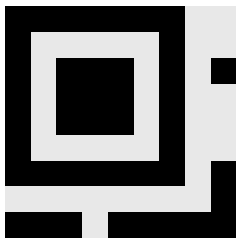
Use the column for your k. Each bit string reads left to right.

number	k = 1	k = 2	k = 3	k = 4	k = 5
0	0	00	000	0000	00000
1	1	01	001	0001	00001
2		10	010	0010	00010
3		11	011	0011	00011
4			100	0100	00100
5			101	0101	00101
6			110	0110	00110
7			111	0111	00111
8				1000	01000
9				1001	01001
10				1010	01010
11				1011	01011
12				1100	01100
13				1101	01101
14				1110	01110
15				1111	01111

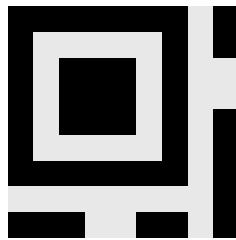
number	k = 1	k = 2	k = 3	k = 4	k = 5
16					10000
17					10001
18					10010
19					10011
20					10100
21					10101
22					10110
23					10111
24					11000
25					11001
26					11010
27					11011
28					11100
29					11101
30					11110
31					11111

Masks

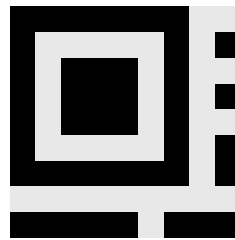
The top-left corner of a QR code (rows and columns 1–9). The 15 format cells — row 9 and column 9 — tell its mask; they are the same for 21 × 21 and 25 × 25. The header carries the mask as 3 bits.



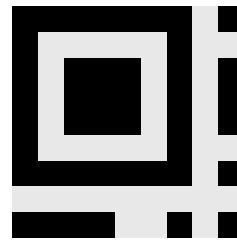
mask 0 · 000



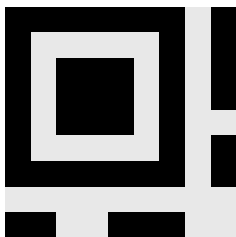
mask 1 · 001



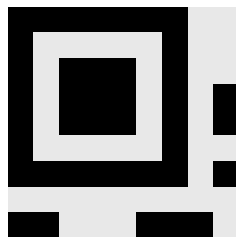
mask 2 · 010



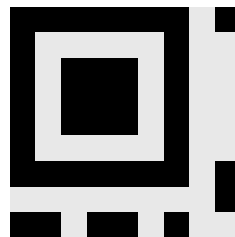
mask 3 · 011



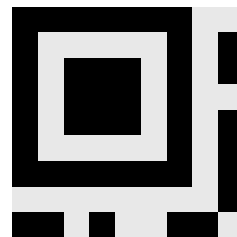
mask 4 · 100



mask 5 · 101



mask 6 · 110



mask 7 · 111

Worksheet

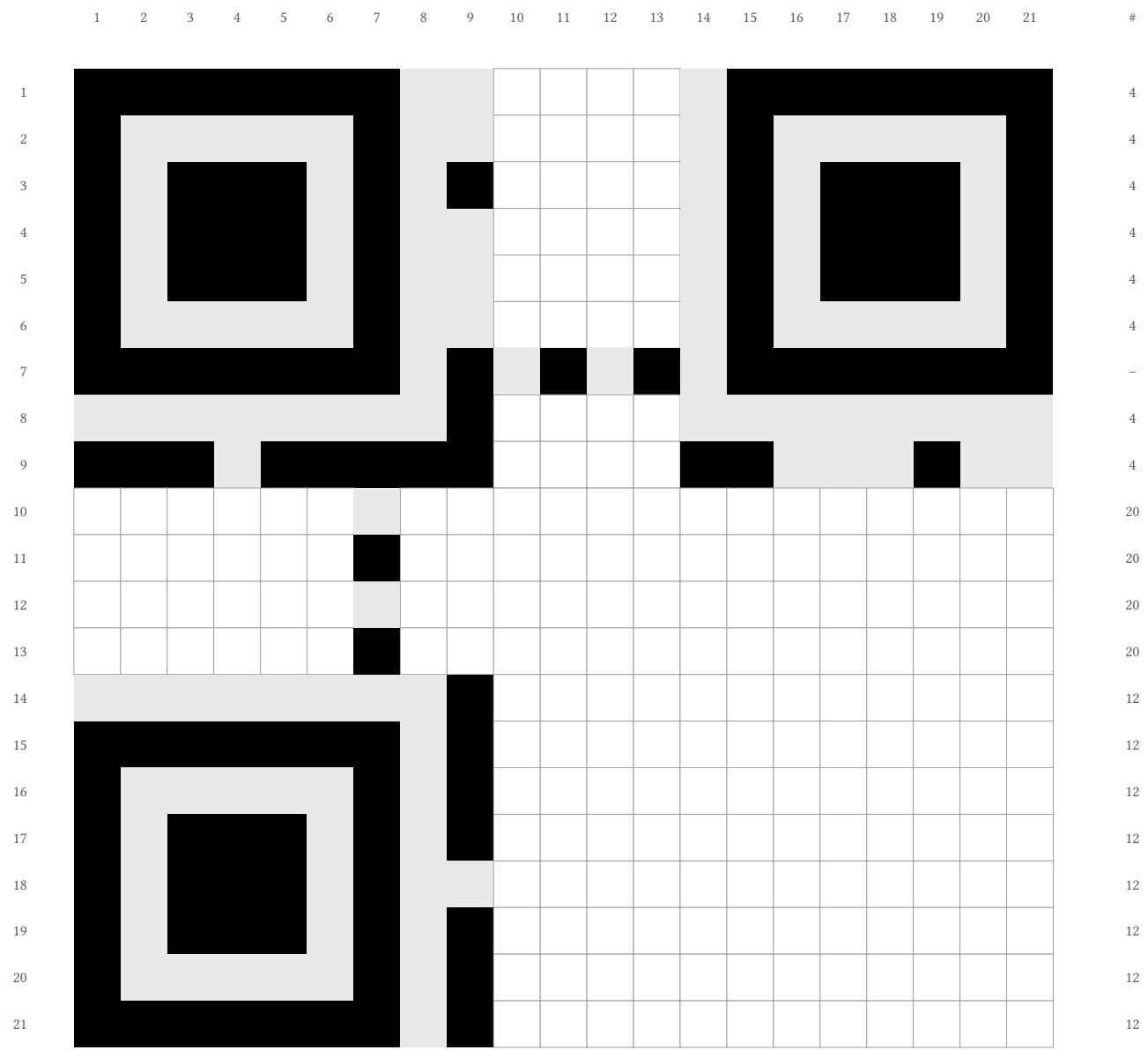
Destroy it when you finish. Dictionary: draw or name each object. Then one box per object after the dictionary: its number above, its bits below.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120
121	122	123	124	125	126	127	128	129	130

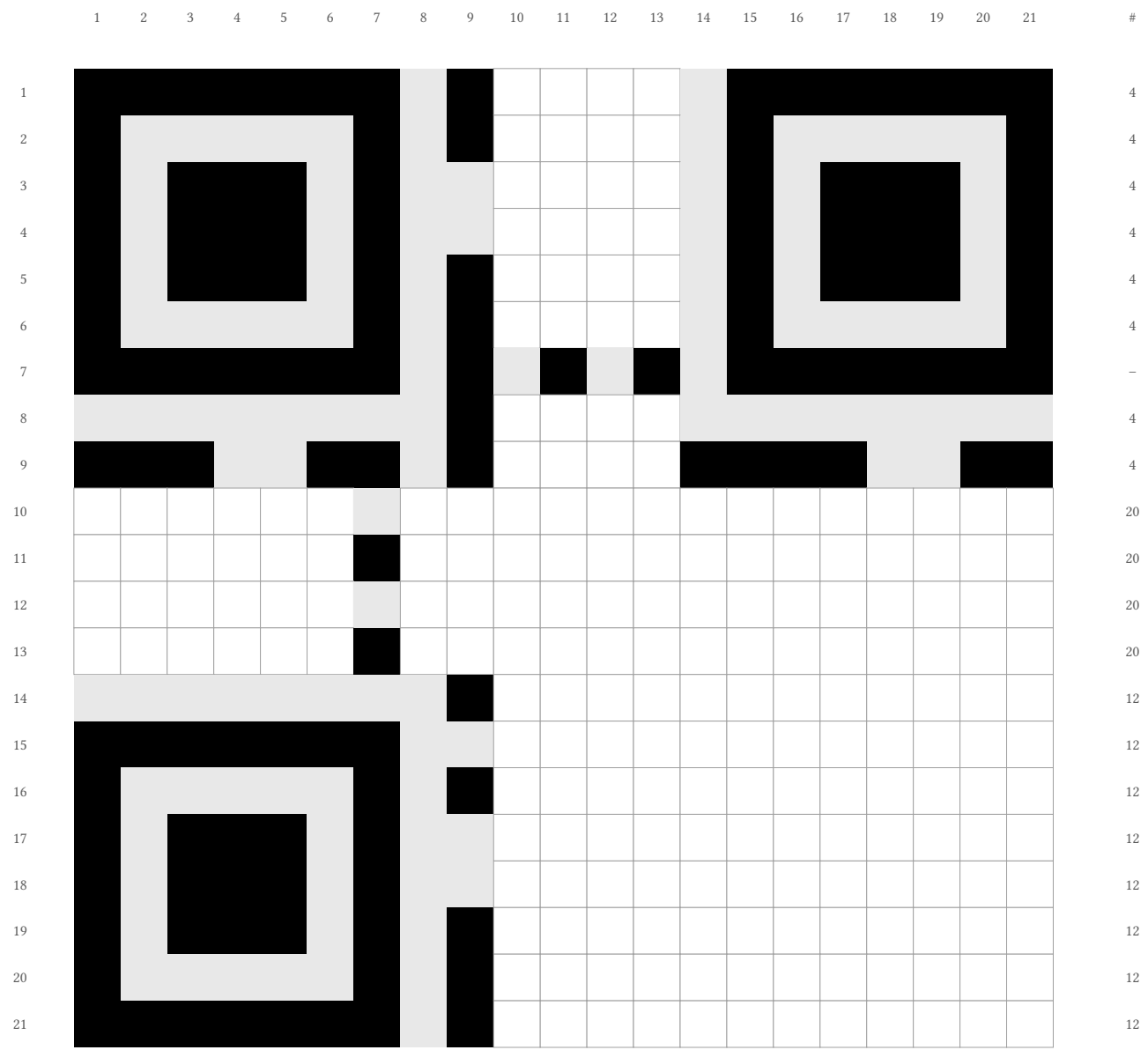
Template 21 × 21 · mask 0

12 words · header 00000 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



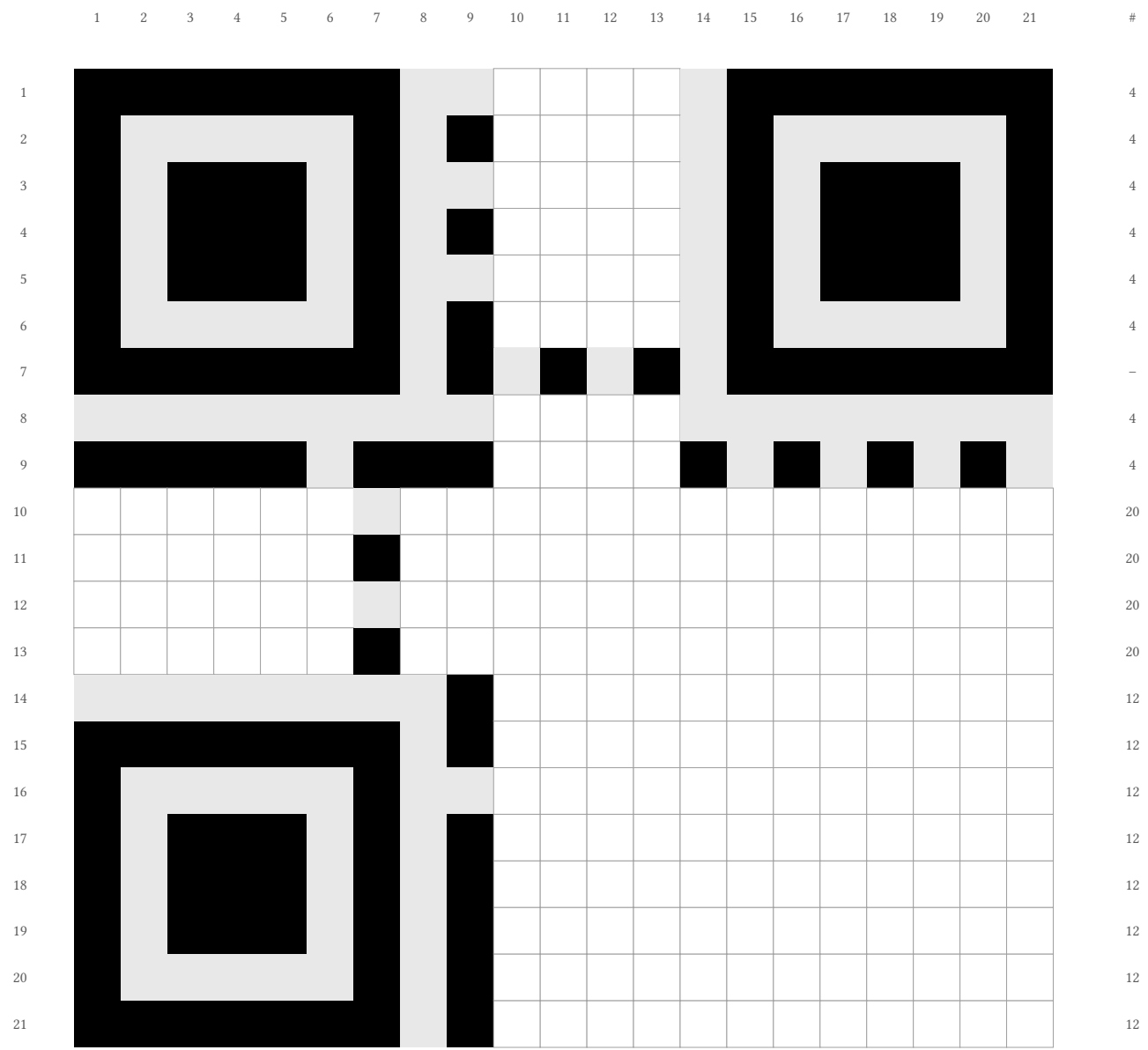
Template 21 × 21 · mask 1

12 words · header 00001 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



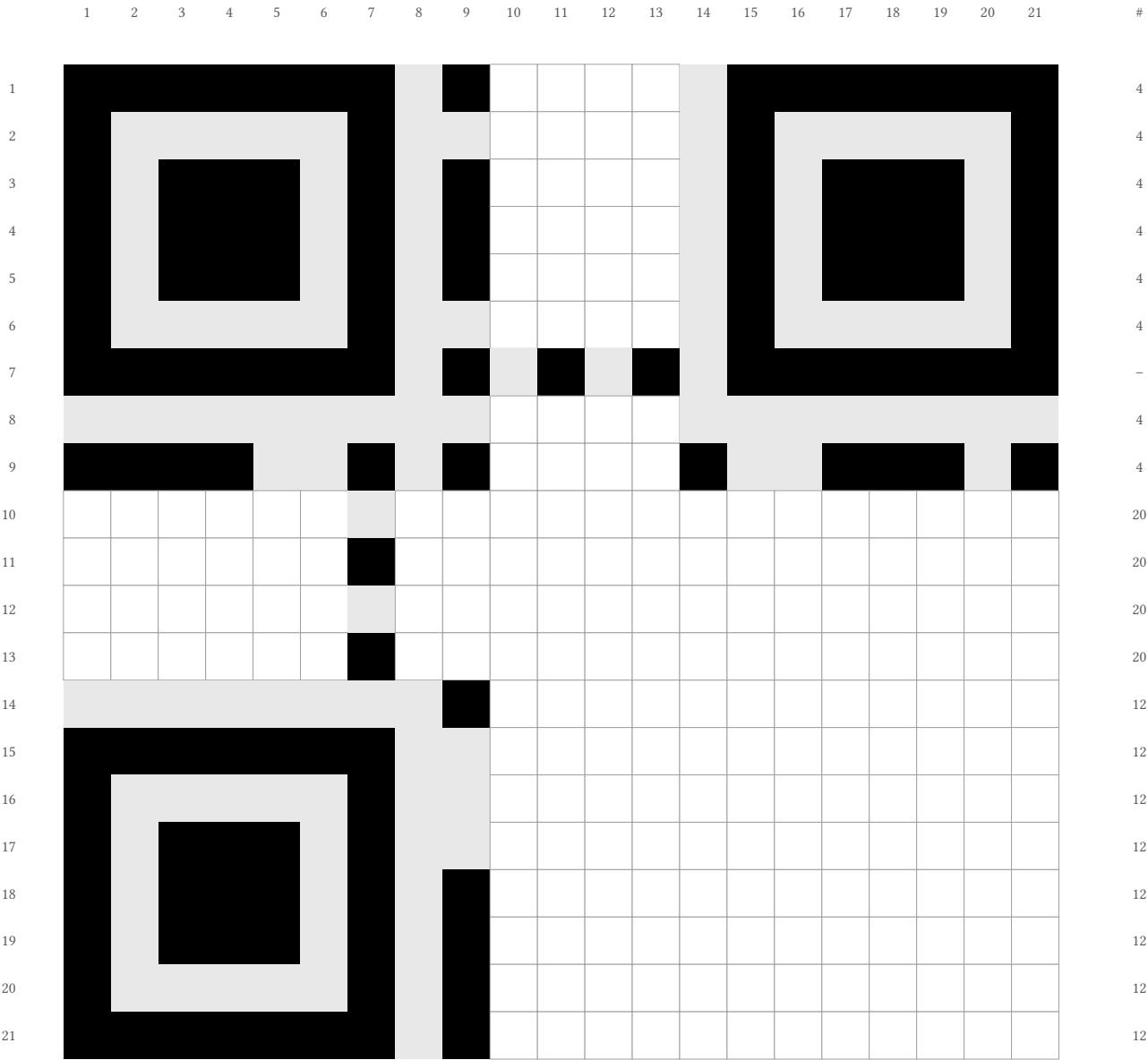
Template 21 × 21 · mask 2

12 words · header 00010 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



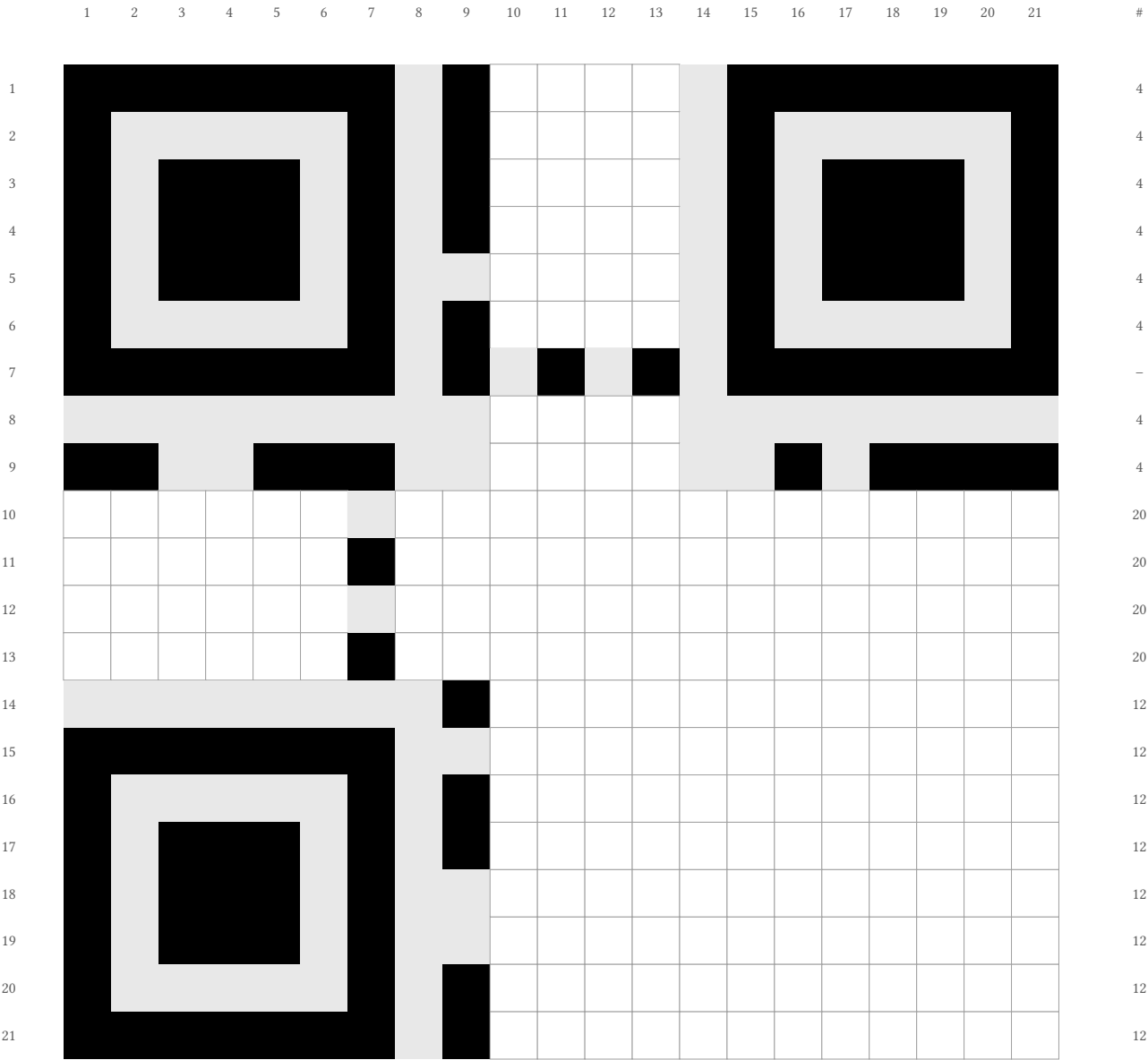
Template 21 × 21 · mask 3

12 words · header 00011 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



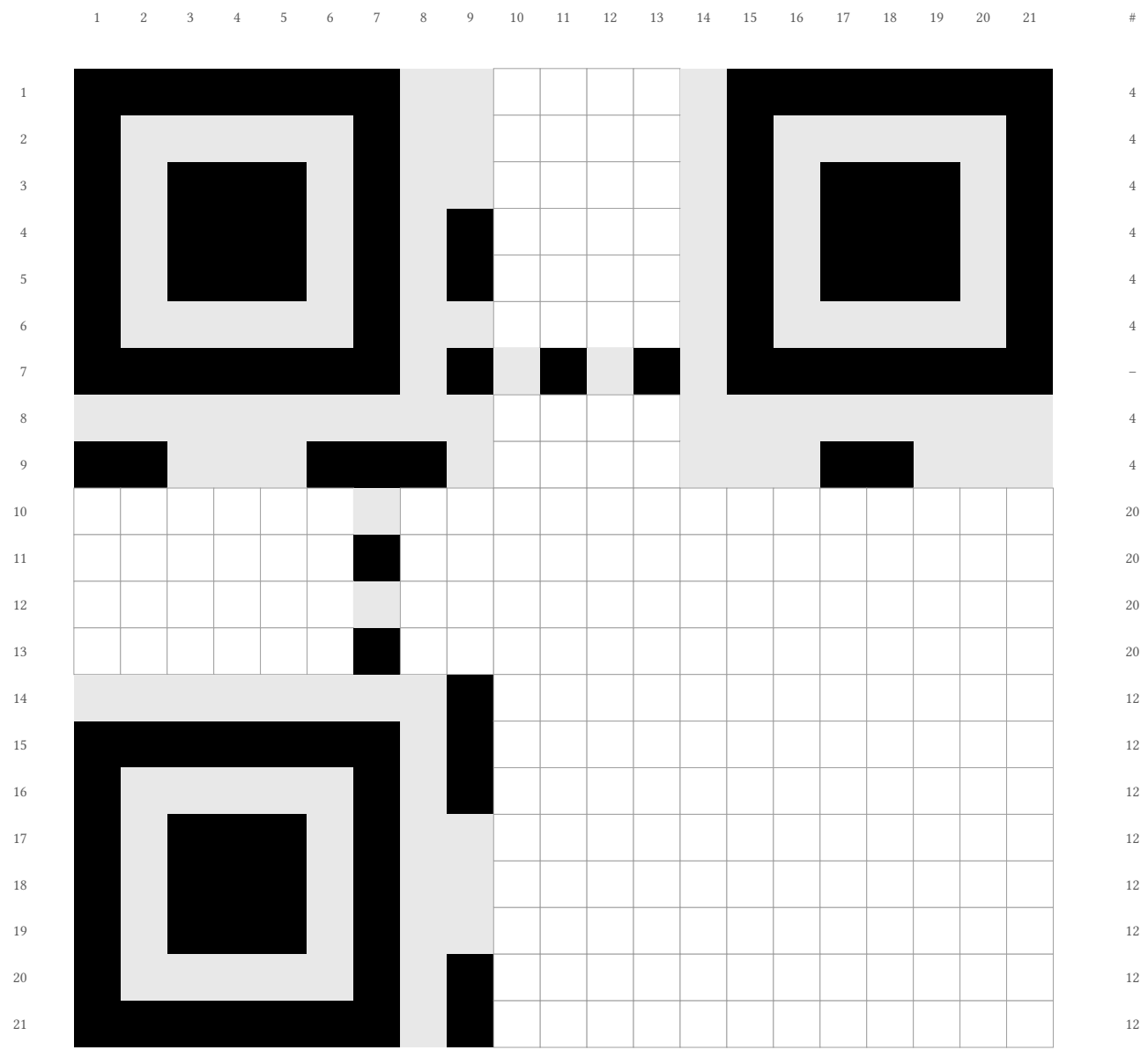
Template 21 × 21 · mask 4

12 words · header 00100 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



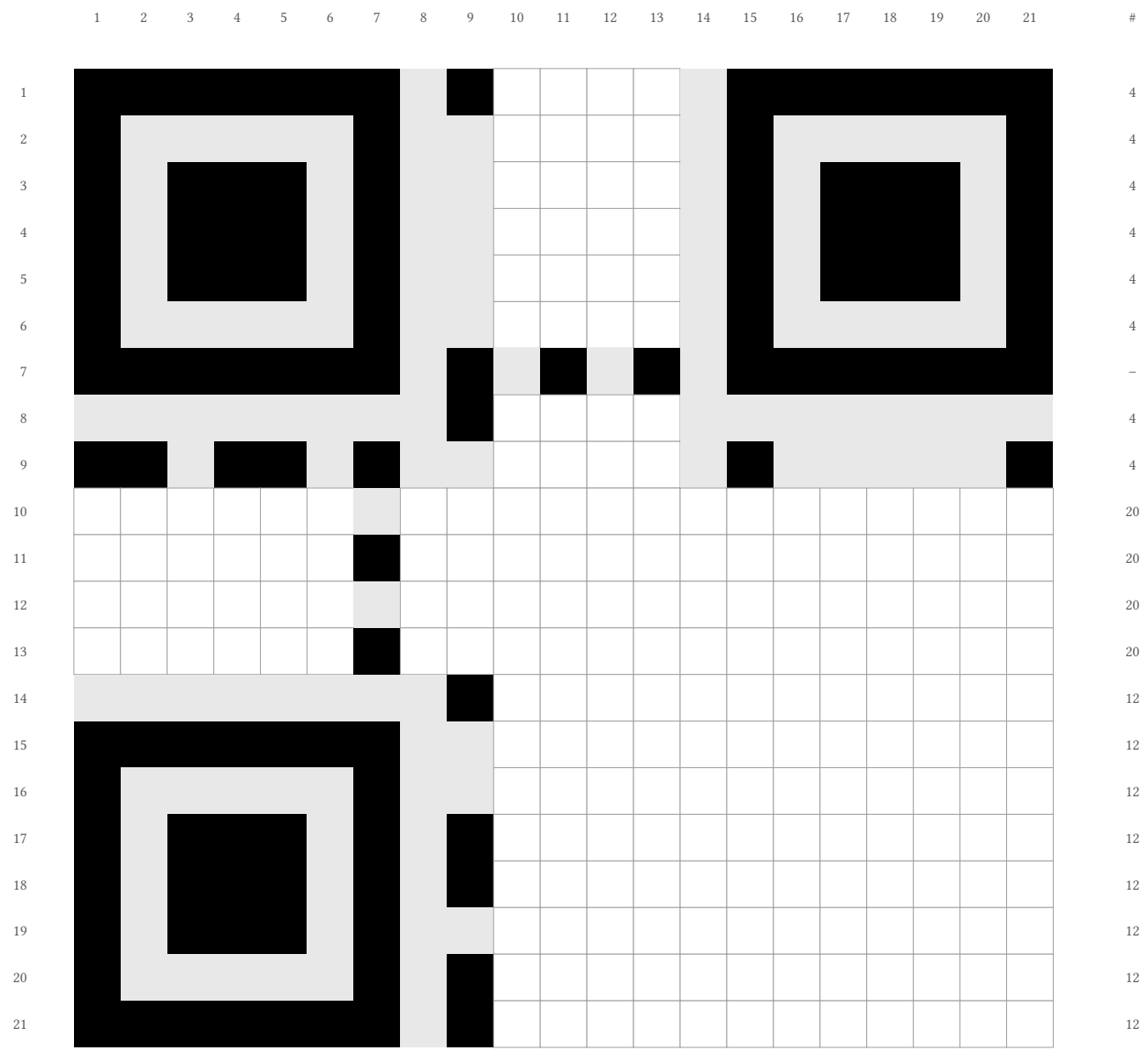
Template 21 × 21 · mask 5

12 words · header 00101 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



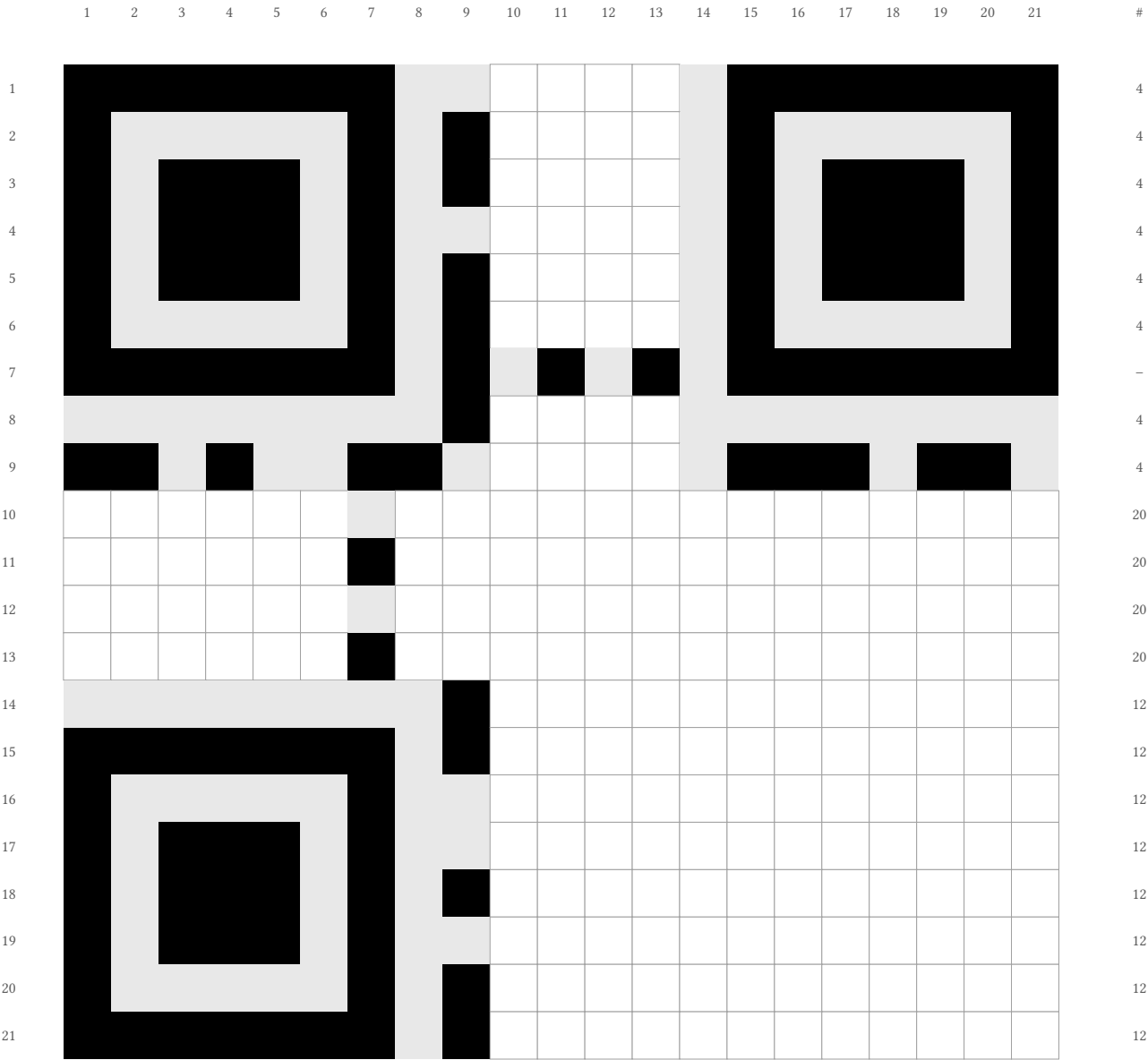
Template 21 × 21 · mask 6

12 words · header 00110 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



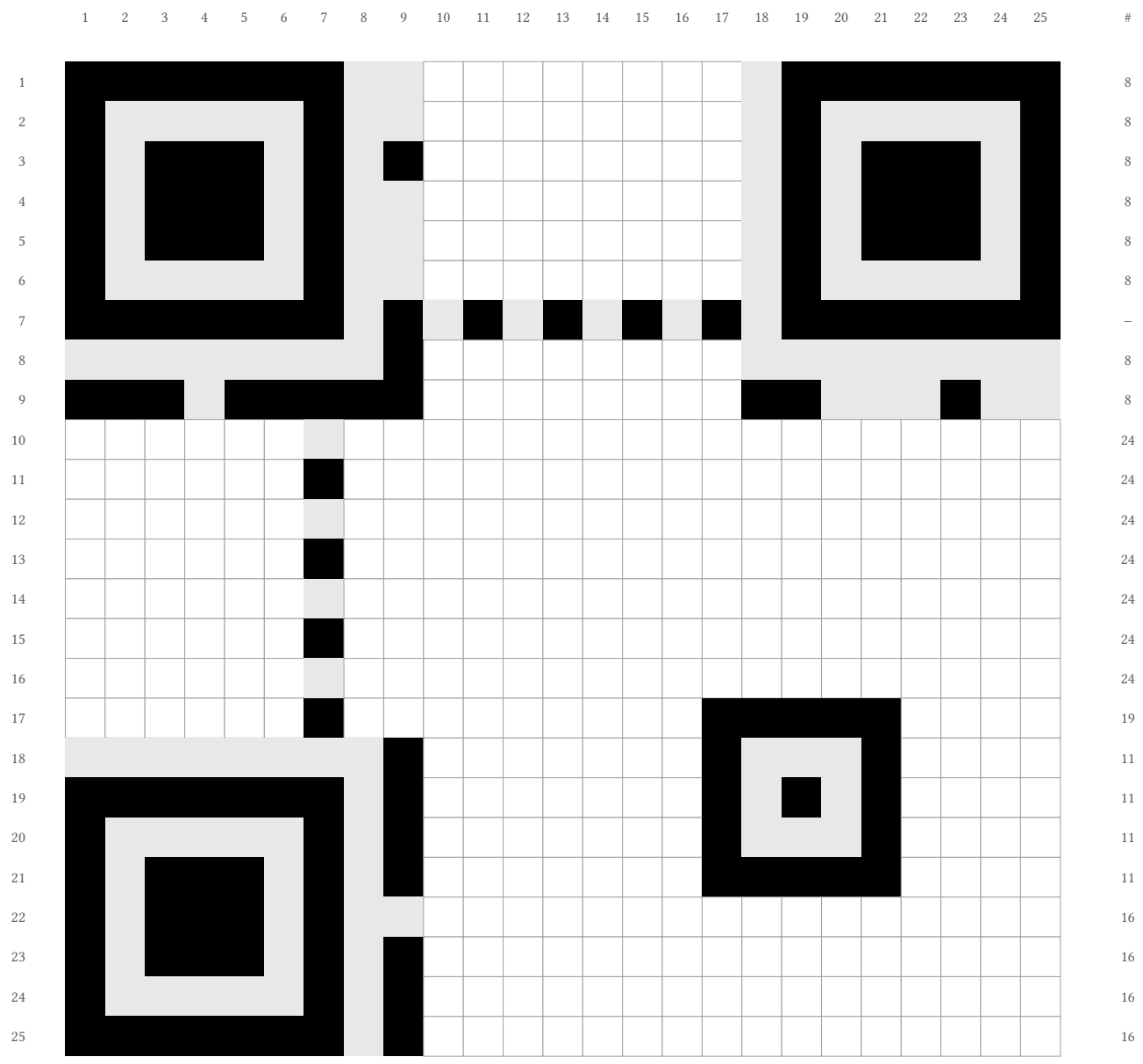
Template 21 × 21 · mask 7

12 words · header 00111 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



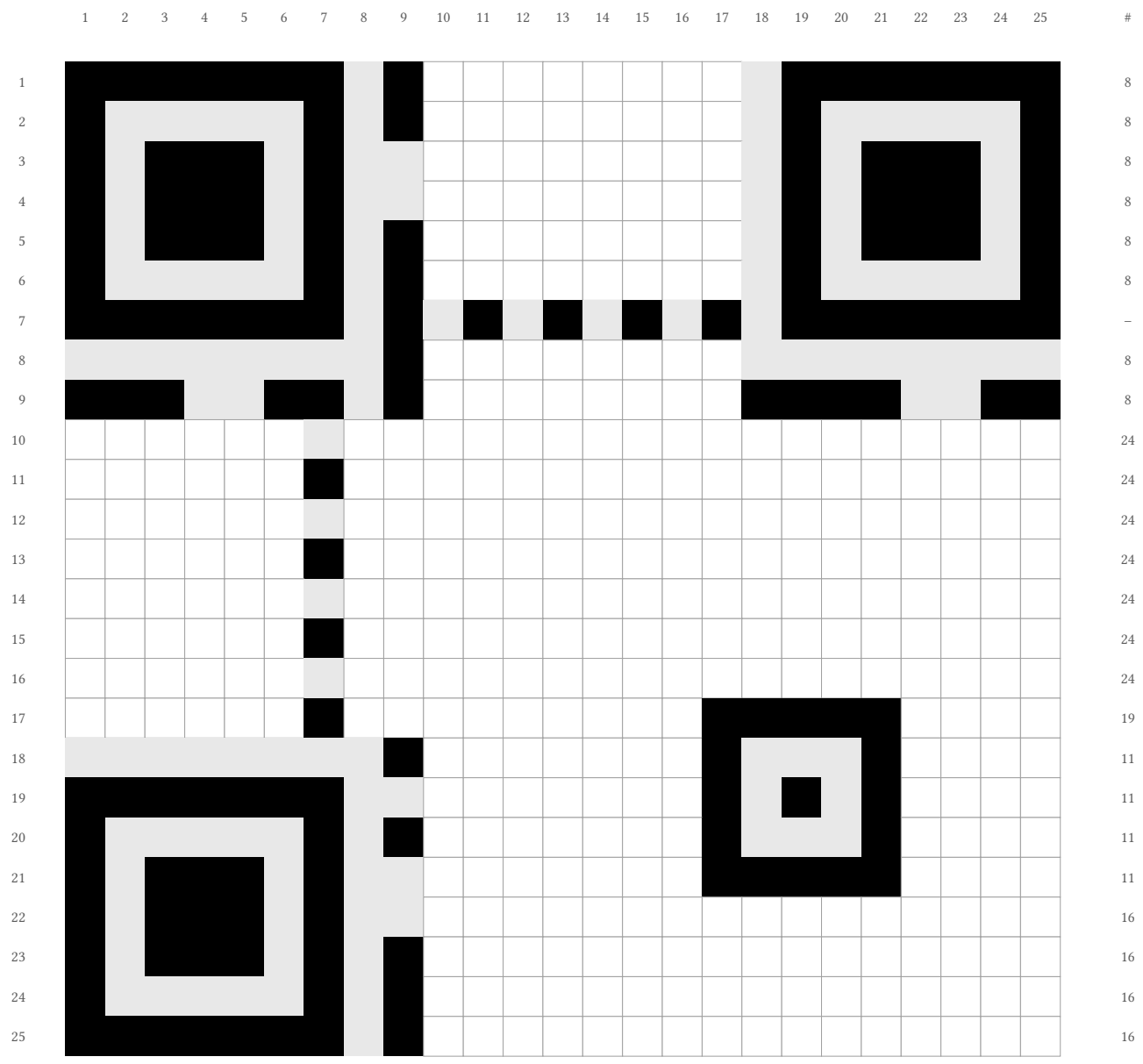
Template 25 × 25 · mask 0

24 words · header 00000 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



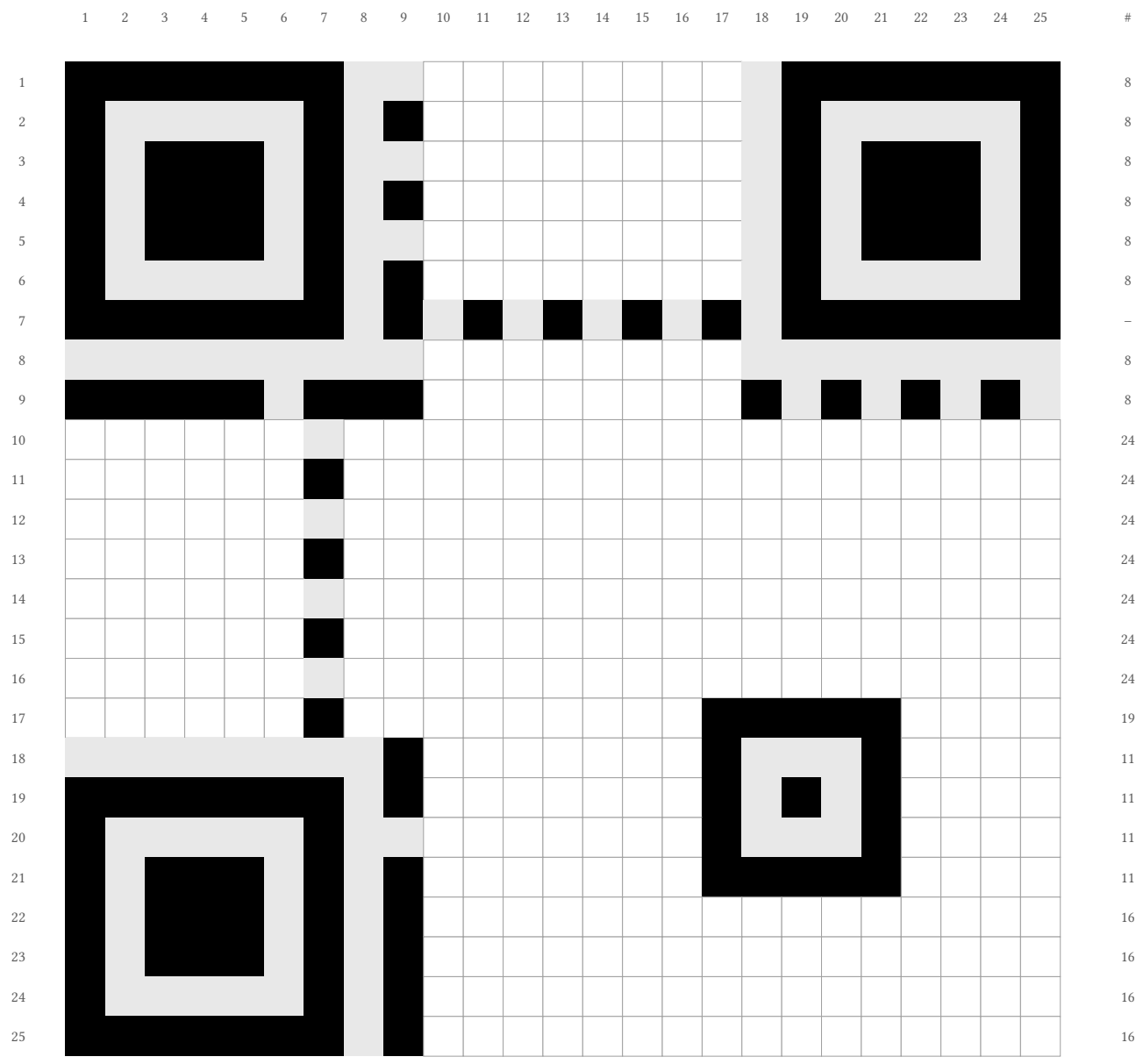
Template 25 × 25 · mask 1

24 words · header 00001 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



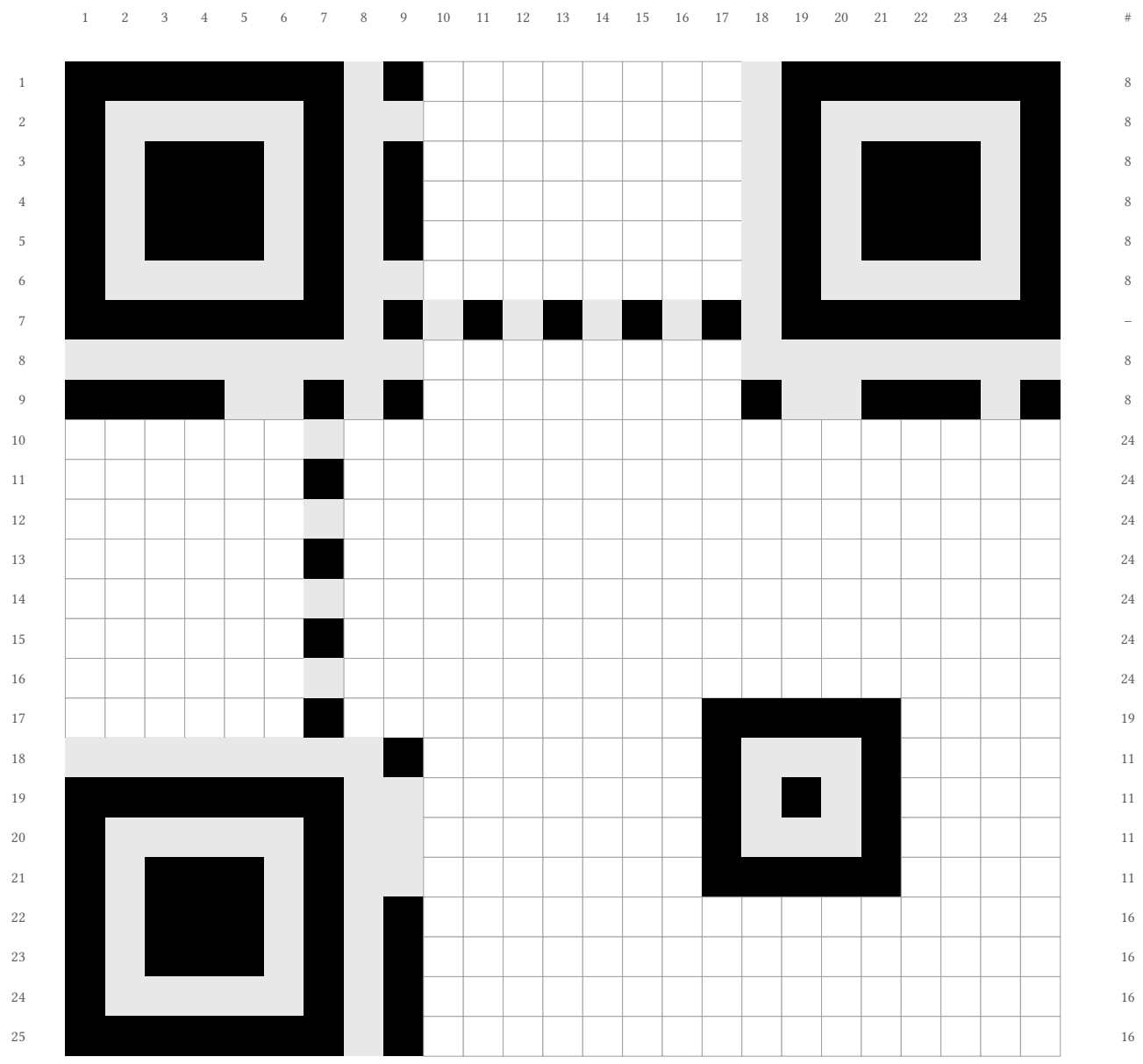
Template 25 × 25 · mask 2

24 words · header 00010 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



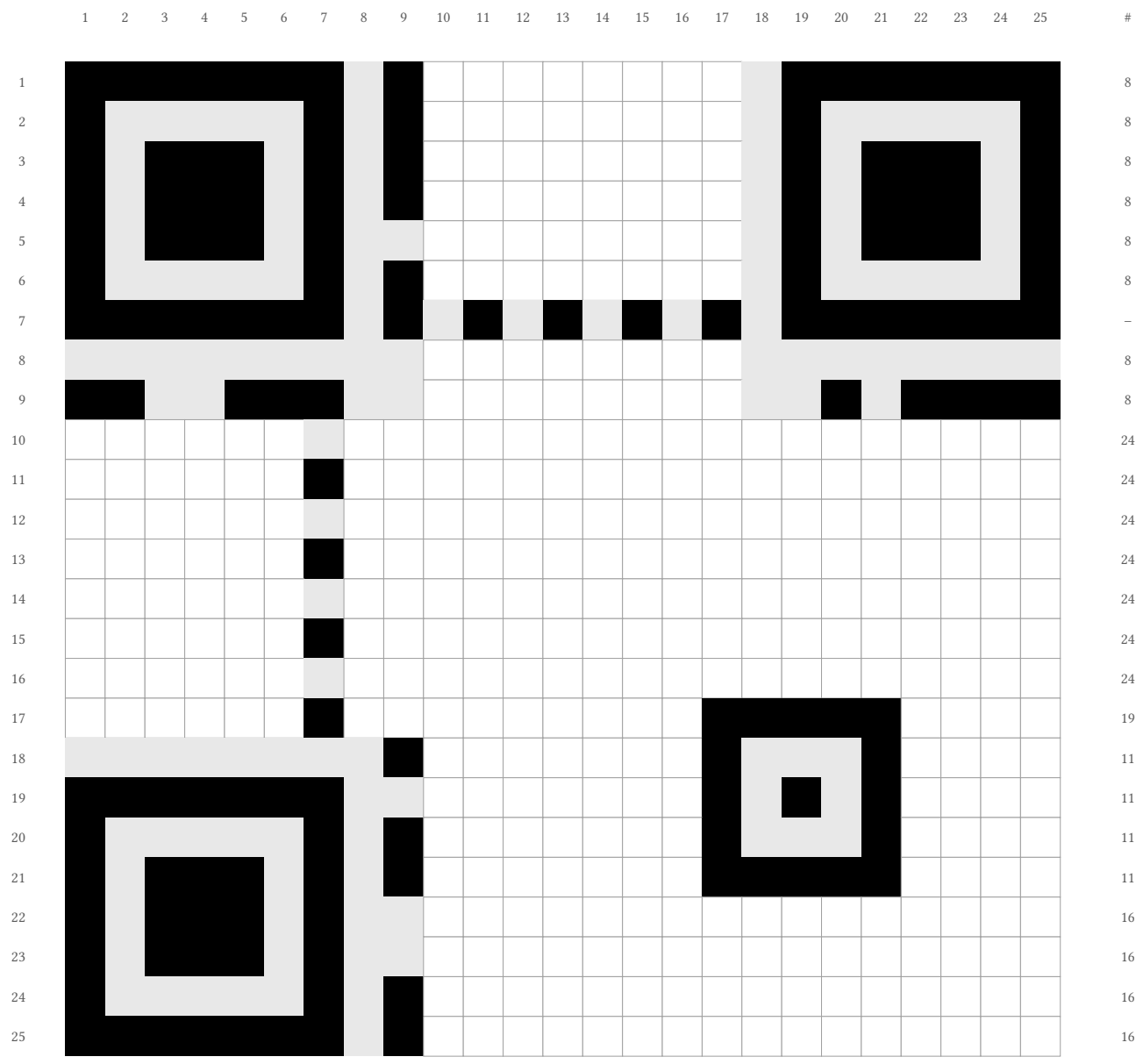
Template 25 × 25 · mask 3

24 words · header 00011 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



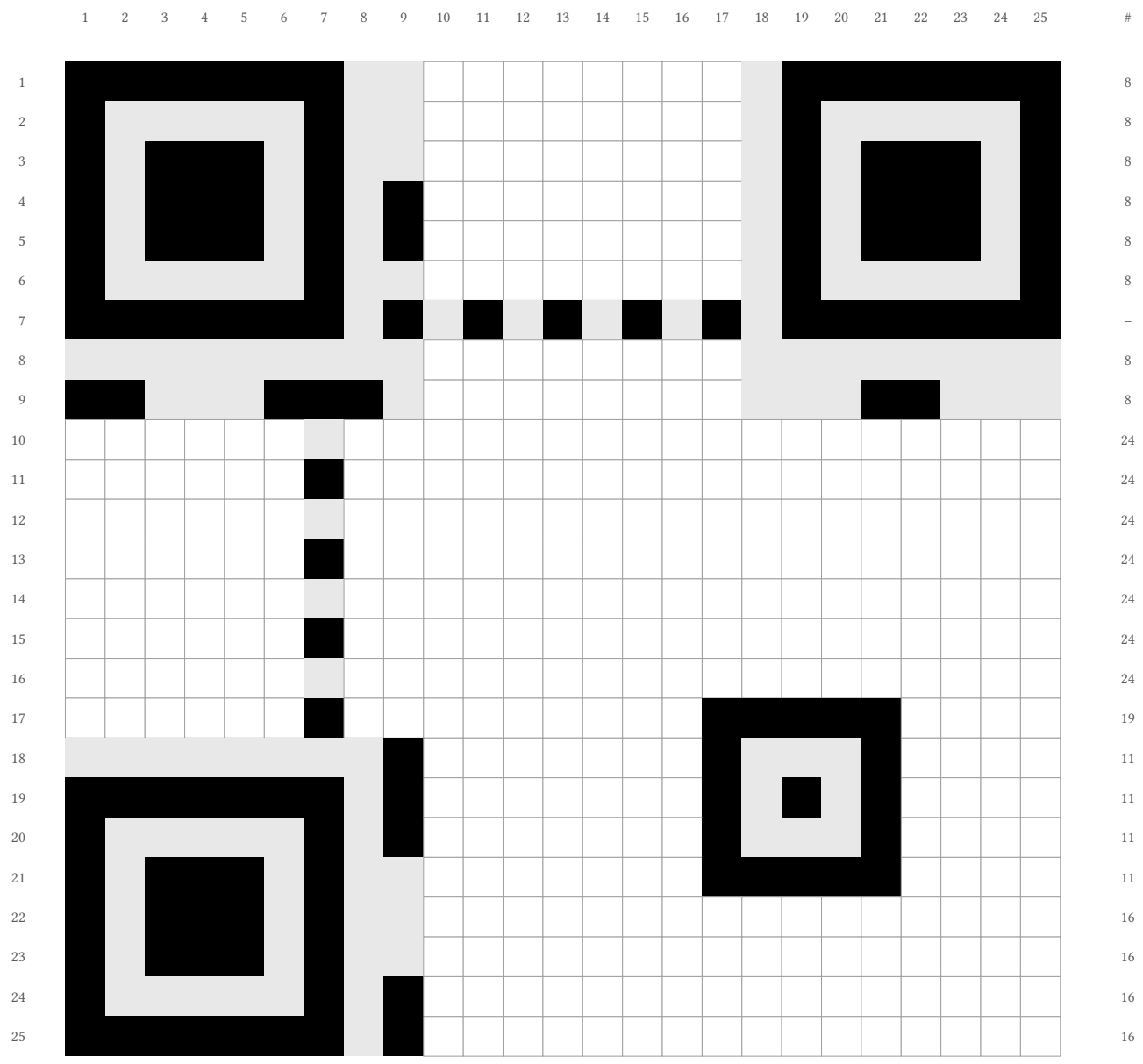
Template 25 × 25 · mask 4

24 words · header 00100 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



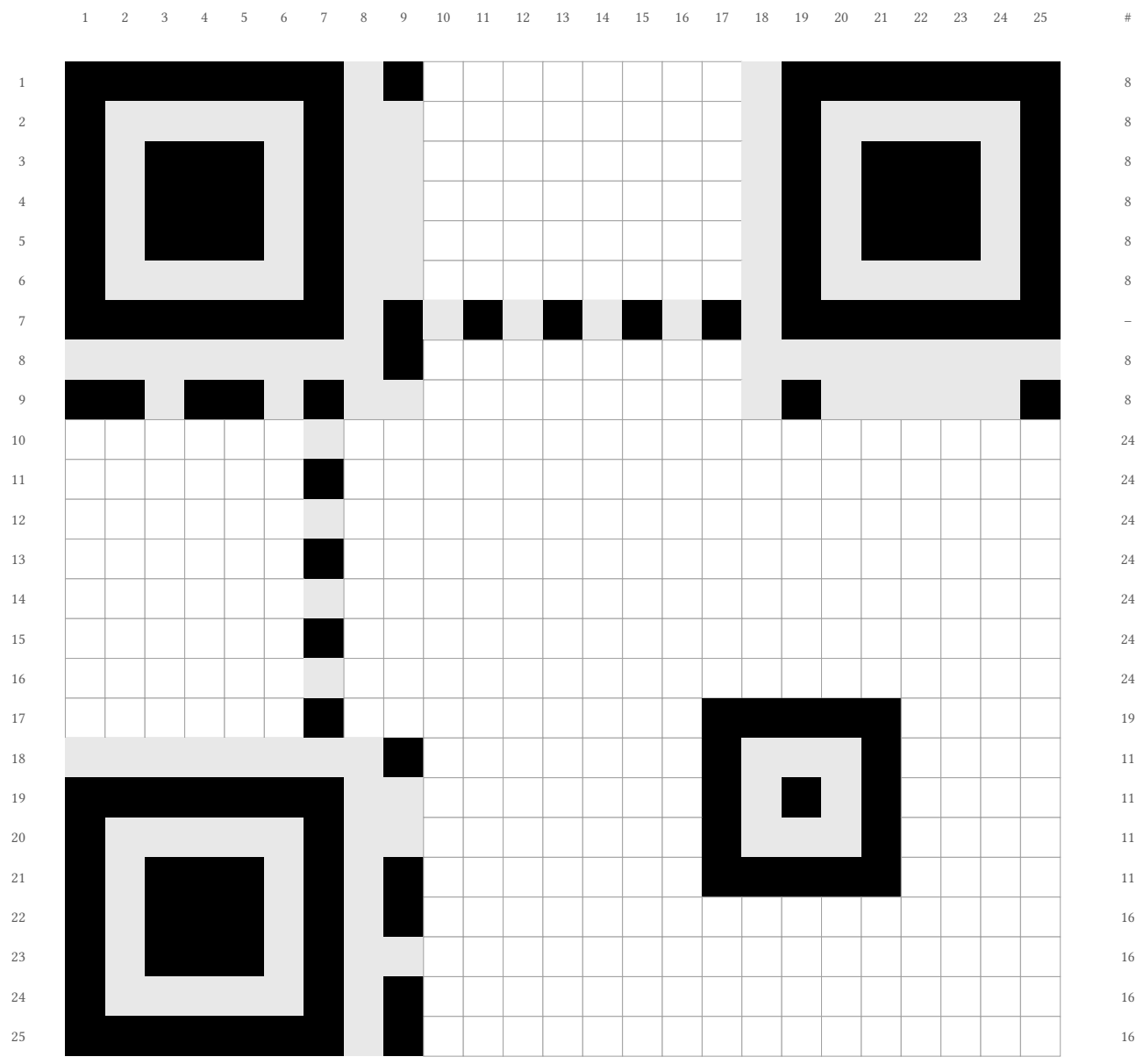
Template 25 × 25 · mask 5

24 words · header 00101 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



Template 25 × 25 · mask 6

24 words · header 00110 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



Template 25 × 25 · mask 7

24 words · header 00111 · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.

